Basics Day 11 — Session 11 (Hours 41–44)

Python Programming (Basic) • File I/O, JSON, Paths & Exception Handling

Session 11 Overview

- Hour 41: Files — reading and writing text
- Hour 42: JSON persistence (stdlib json)
- Hour 43: Directories + paths (pathlib preferred)
- Hour 44: Exception handling (full Basics treatment)

Hour 41: Files — Reading and Writing Text

Learning Outcomes

- Open files using `with open()`
- Read all text, iterate lines, and write lines to a file
- Understand how the context manager closes files safely

Why Files Matter

Programs Forget Everything Without Files

```
# Without files — data is gone when the program exits
contacts = ["Alice|555-1234", "Bob|555-5678"]
# ... user adds Carol at runtime ...
# Program exits → Carol is lost forever
```

With Files — Data Survives Between Runs

```
# Save to disk
with open("contacts.txt", "w") as f:
    for entry in contacts:
        f.write(entry + "\n")

# Next run — load it back
with open("contacts.txt", "r") as f:
    contacts = [line.strip() for line in f]
```

The `with` Statement — Context Manager Basics

What Problem Does `with` Solve?

```
# Old-style (fragile) — what if an error happens before close()?
f = open("contacts.txt", "r")
data = f.read()     # if this raises, f.close() is never called!
f.close()           # file handle leaks → resource exhaustion over time

# with open() — guaranteed close, even on error
with open("contacts.txt", "r") as f:
    data = f.read()
# f is automatically closed here — no matter what happened inside
```

The Three Parts

```
with open("contacts.txt", "r") as f:
#    ^^^^ ^^^^^^^^^^^^^^^^  ^^^  ^
#    |    filename           |   variable name bound to the file object
#    |                      mode: "r" = read, "w" = write, "a" = append
```

File Modes

Common Modes at a Glance

Mode	Meaning	File must exist?	Overwrites?
`"r"`	Read (text)	Yes — raises `FileNotFoundError`	—
`"w"`	Write (text)	No — creates it	Yes — truncates!
`"a"`	Append (text)	No — creates it	No — adds to end
`"x"`	Exclusive create	No — raises if it exists	—

Warning: `"w"` Overwrites!

```
# First call — writes "Hello\n" to file
with open("notes.txt", "w") as f:
    f.write("Hello\n")

# Second call — ERASES "Hello\n", replaces with "World\n"
```

Reading a File — Three Patterns

Pattern 1: `read()` — Entire File as One String

```
with open("contacts.txt", "r") as f:
    content = f.read()          # one big string including all newlines
print(content)
```

Pattern 2: `readlines()` — List of Lines (with `\n`)

```
with open("contacts.txt", "r") as f:
    lines = f.readlines()       # ["Alice|555-1234\n", "Bob|555-5678\n"]

for line in lines:
    print(line.strip())         # .strip() removes the trailing \n
```

Pattern 3: Iterate Directly (Most Memory-Efficient)

```
with open("contacts.txt", "r") as f:
```

Writing a File

`write()` — Write a String

```
with open("contacts.txt", "w") as f:
    f.write("Alice|555-1234\n")   # you must include the newline yourself!
    f.write("Bob|555-5678\n")
```

`writelines()` — Write an Iterable of Strings

```
entries = ["Alice|555-1234\n", "Bob|555-5678\n", "Carol|555-9999\n"]

with open("contacts.txt", "w") as f:
    f.writelines(entries)         # entries must already contain "\n"
```

Saving a List (Pattern You Will Use in Lab)

```
contacts = ["Alice|555-1234", "Bob|555-5678", "Carol|555-9999"]
```

File Paths and Working Directory

Relative vs Absolute

```
# Relative path — relative to where you RUN the script from
with open("contacts.txt", "r") as f: ...      # same folder as CWD
with open("data/contacts.txt", "r") as f: ... # subfolder of CWD

# Absolute path — always the same regardless of where you run from
with open("/home/user/project/contacts.txt", "r") as f: ...
```

Finding Your Working Directory

```
import os
print(os.getcwd())   # prints e.g. /home/user/project
```

Checking Whether a File Exists

```
import os
```

Demo: Shopping List — Write Then Read

Watch For:

- `"w"` mode creates the file
- Newline added manually in `write()`
- `"r"` mode reads back exactly what was written
- `.strip()` removes trailing `\n` on each line

```
# --- Write the shopping list ---
items = ["apples", "bread", "milk", "eggs", "cheese"]

with open("shopping.txt", "w") as f:
    for item in items:
        f.write(item + "\n")

print("Shopping list saved.")

# --- Read it back and print ---
```

Lab: Save and Load (Text) — 30 min

Task

Build a program that saves contacts (or tasks) to a text file and loads them back.

Step 1 — Define your format:

```
Alice|555-1234
Bob|555-5678
Carol|555-9999
```

Step 2 — Write a `save_contacts()` function:

```python
def save_contacts(contacts: list, filename: str) -> None:
    with open(filename, "w") as f:
        for entry in contacts:
            f.write(entry + "\n")
```

Step 3 — Write a `load_contacts()` function:

Lab: Save and Load (Text) — continued

Step 4 — Add a menu and test:

```
import os

FILENAME = "contacts.txt"

contacts = []

# Load on startup (only if file exists)
if os.path.exists(FILENAME):
    contacts = load_contacts(FILENAME)
    print(f"Loaded {len(contacts)} contacts.")

while True:
    cmd = input("add / list / quit: ").strip().lower()
    if cmd == "add":
        name  = input("  Name: ").strip()
        phone = input("  Phone: ").strip()
        contacts.append(f"{name}|{phone}")
        save_contacts(contacts, FILENAME)
        print("Saved.")
    elif cmd == "list":
        for entry in contacts:
            parts = entry.split("|")
            print(f"  {parts[0]} — {parts[1]}")
```

Common Pitfalls — Hour 41

Pitfall 1: Wrong Working Directory

```
# You think your script is in /home/user/project/
# You double-click from the Desktop → CWD is /home/user/Desktop
# Result: FileNotFoundError — "contacts.txt" not where you expect

# Fix: always run your script from a known location
# Terminal: cd /home/user/project && python main.py
# Or use absolute paths (Hour 43: pathlib fixes this properly)
```

Pitfall 2: Forgetting the Newline

```
# Bug — all entries on one line: "AliceBobCarol"
with open("contacts.txt", "w") as f:
    for entry in contacts:
        f.write(entry)          # no "\n"!

# Fix
```

Common Pitfalls — Hour 41 (continued)

Pitfall 3: Using `"w"` When You Mean `"a"`

```
# Bug: each run overwrites the file — history is lost
contacts = load_contacts("contacts.txt")
contacts.append("Dave|555-4321")

with open("contacts.txt", "w") as f:   # wipes file first!
    f.write("Dave|555-4321\n")          # only Dave remains after restart

# Fix: either write the full list or use "a" intentionally
with open("contacts.txt", "w") as f:
    for entry in contacts:              # write the full updated list
        f.write(entry + "\n")
```

Pitfall 4: Not Closing The File (No `with`)

```
# Bug — no guarantee the file is flushed or closed
f = open("contacts.txt", "w")
```

Quick Check — Hour 41

Exit Ticket Question: Why is using `with` recommended for file handling?

Model Answer: "The `with` statement uses Python's **context manager** protocol to guarantee that the file is closed when the block exits — whether the code inside the block ran successfully or raised an exception. Without `with`, you must call `f.close()` manually, and if an exception is raised before `close()` is reached, the file handle leaks. A leaked file handle can prevent other processes from accessing the file, exhaust the operating system's file descriptor limit, and may leave data un-flushed to disk. Using `with open(...)` is the idiomatic, safe pattern for all file I/O in Python."

Hour 42: JSON Persistence (stdlib `json`)

Learning Outcomes

- Serialize Python dicts/lists to a JSON file with `json.dump()`
- Load JSON back into Python with `json.load()`
- Understand which Python types map to JSON
- Convert a simple object to/from a dict for JSON storage

Why JSON?

Plain Text Files Have Limits

```
# Text format: one contact per line — how do you store extra fields?
"Alice|555-1234"                 # simple
"Alice|555-1234|alice@ex.com"    # add email — fine so far
"Alice|555-1234|alice@ex.com|notes: likes cats, no calls before 9am"
# notes field contains "|" → parsing breaks
```

JSON Handles Structure Natively

```
import json

contacts = [
    {"name": "Alice", "phone": "555-1234", "email": "alice@ex.com"},
    {"name": "Bob",   "phone": "555-5678", "email": "bob@ex.com"},
]

# Save
```

What JSON Can Store

Python → JSON Type Mapping

Python Type	JSON Type	Example
`dict`	object `{}`	`{"name": "Alice"}`
`list`	array `[]`	`["a", "b", "c"]`
`str`	string	`"hello"`
`int` / `float`	number	`42`, `3.14`
`True` / `False`	`true` / `false`	`true`
`None`	`null`	`null`
custom object	❌ not directly	convert to `dict` first
`tuple`	array `[]`	becomes a list on load

`json.dump` and `json.load`

Saving to File — `json.dump()`

```
import json

data = {
    "app": "Contact Manager",
    "version": 1,
    "contacts": [
        {"name": "Alice", "phone": "555-1234"},
        {"name": "Bob",   "phone": "555-5678"},
    ]
}

with open("data.json", "w") as f:
    json.dump(data, f, indent=2)   # indent=2 for human-readable output
```

Loading from File — `json.load()`

`indent` and Pretty-Printing

Without `indent` — Compact (Hard to Read)

```
json.dump(data, f)
# {"app": "Contact Manager", "version": 1, "contacts": [{"name": "Alice", ...}]}
```

With `indent=2` — Readable

```
{
  "app": "Contact Manager",
  "version": 1,
  "contacts": [
    {
      "name": "Alice",
      "phone": "555-1234"
    },
    {
      "name": "Bob",
      "phone": "555-5678"
```

Converting Objects ↔️ Dicts

Why Custom Objects Need Conversion

```
import json

class Contact:
    def __init__(self, name: str, phone: str) -> None:
        self.name  = name
        self.phone = phone

c = Contact("Alice", "555-1234")
json.dumps(c)
# TypeError: Object of type Contact is not JSON serializable
```

Add `to_dict()` and `from_dict()` Class Methods

```
class Contact:
    def __init__(self, name: str, phone: str) -> None:
        self.name  = name
```

Saving and Loading Objects (Full Pattern)

Save a List of Contact Objects

```
import json

def save_contacts(contacts: list, filename: str) -> None:
    data = [c.to_dict() for c in contacts]
    with open(filename, "w") as f:
        json.dump(data, f, indent=2)
```

Load and Reconstruct Contact Objects

```
def load_contacts(filename: str) -> list:
    with open(filename, "r") as f:
        data = json.load(f)
    return [Contact.from_dict(d) for d in data]
```

Usage Pattern in `main()`

Demo: Dump and Load a Dict — Watch the Live Build

Watch For:

- `json.dump()` writes the file; `json.load()` reads it back
- Python types survive the round-trip intact
- `indent=2` makes the file readable

```python
import json, os

# Build data
data = {
    "contacts": [
        {"name": "Alice", "phone": "555-1234"},
        {"name": "Bob",   "phone": "555-5678"},
    ],
    "count": 2
}

# Save
with open("data.json", "w") as f:
    json.dump(data, f, indent=2)
print("Saved data.json")
```

Lab: Save and Load (JSON) — 30 min

Task

Add JSON persistence to your contact manager from Hour 41.

Step 1 — Add `to_dict()` to your `Contact` class:

```
def to_dict(self) -> dict:
    return {"name": self.name, "phone": self.phone}
```

Step 2 — Add a standalone `from_dict()` function (Basics-friendly):

```
def contact_from_dict(data: dict) -> Contact:
    return Contact(data["name"], data["phone"])
```

Step 3 — Save and load functions:

```
def save_contacts(contacts: list, filename: str) -> None:
```

Lab: Save and Load (JSON) — continued

Step 4 — Wire up startup and shutdown:

```
import json, os

FILENAME = "data.json"
contacts = []

if os.path.exists(FILENAME):
    contacts = load_contacts(FILENAME)
    print(f"Loaded {len(contacts)} contacts from {FILENAME}")

# ... menu loop (add / list / quit) ...

save_contacts(contacts, FILENAME)
print("Changes saved.")
```

Completion Criteria

- [ ] Program saves `data.json` after any change

Common Pitfalls — Hour 42

Pitfall 1: `JSONDecodeError` From an Empty or Corrupt File

```
# Bug: file exists but is empty (e.g., write failed mid-way)
with open("data.json", "r") as f:
    data = json.load(f)   # json.JSONDecodeError: Expecting value: line 1 col 1

# Fix (Hour 44 gives the full pattern): check file size or use try/except
import os

if os.path.exists("data.json") and os.path.getsize("data.json") > 0:
    with open("data.json", "r") as f:
        data = json.load(f)
else:
    data = []
```

Pitfall 2: Trying to Dump a Custom Object Directly

```
c = Contact("Alice", "555-1234")
```

Common Pitfalls — Hour 42 (continued)

Pitfall 3: `tuple` Becomes `list` After JSON Round-Trip

```
import json

original = (1, 2, 3)
encoded  = json.dumps(original)   # "[1, 2, 3]"
decoded  = json.loads(encoded)    # [1, 2, 3]  ← list, not tuple!

print(type(original))   # <class 'tuple'>
print(type(decoded))    # <class 'list'>
```

> ⚠️ **JSON has no tuple type.** Tuples are encoded as JSON arrays and come back as Python lists. If your code checks `isinstance(x, tuple)` after a load, it will break.

Pitfall 4: `set` Cannot Be Serialized

Quick Check — Hour 42

Exit Ticket Question: Why can't `json.dump()` write a custom object unless you convert it first?

Model Answer: "The `json` module only knows how to serialize a fixed set of built-in Python types: `dict`, `list`, `str`, `int`, `float`, `bool`, and `None`. It has no idea what your custom class (`Contact`, `Task`, etc.) is or which attributes are meaningful to save. You must explicitly convert the object to one of those supported types — typically a `dict` — before passing it to `json.dump()`. The reverse is also true: `json.load()` only produces plain Python dicts and lists, so you need a reconstruction step (like `from_dict()`) to turn the loaded data back into your custom objects."

Hour 43: Directories + Paths (pathlib Preferred)

Learning Outcomes

- Use `pathlib.Path` to build portable file paths
- Create a `data/` directory safely with `mkdir(exist_ok=True)`
- List directory contents using `iterdir()` and `glob()`
- Understand the difference between relative and absolute paths

Why `pathlib`?

The Old Way — String Concatenation Is Fragile

```
import os

# Different path separators on Windows vs Unix:
path = "data" + "/" + "contacts.json"   # error-prone and hard to read
path = os.path.join("data", "contacts.json")   # better, but verbose

# Does this file exist?
if os.path.exists(os.path.join("data", "contacts.json")):
    ...
```

The `pathlib` Way — Clean and Portable

```
from pathlib import Path

path = Path("data") / "contacts.json"   # "/" operator builds paths
print(path)            # data/contacts.json  (correct separator on all OS)
```

Path Anatomy

Exploring a `Path` Object

```
from pathlib import Path

p = Path("/home/user/project/data/contacts.json")

print(p.name)      # contacts.json     — filename with extension
print(p.stem)      # contacts          — filename without extension
print(p.suffix)    # .json             — just the extension
print(p.parent)    # /home/user/project/data  — the containing folder
print(p.parts)     # ('/', 'home', 'user', 'project', 'data', 'contacts.json')

# Check properties
print(p.is_file())    # True if a file exists at this path
print(p.is_dir())     # True if it is a directory
print(p.exists())     # True if either a file or dir exists here
```

Building Paths With `/`

Relative vs Absolute Paths

The Difference

```
from pathlib import Path

# Relative — interpreted from the current working directory (CWD)
rel = Path("data") / "contacts.json"
print(rel)            # data/contacts.json
print(rel.is_absolute())  # False

# Absolute — the full path from the root of the filesystem
abs_ = Path("/home/user/project/data/contacts.json")
print(abs_.is_absolute())  # True

# Convert relative → absolute (resolves against CWD)
print(rel.resolve())  # /home/user/project/data/contacts.json
```

Why Relative Paths Can Surprise You

Creating Directories Safely

`mkdir()` — Create a Directory

```
from pathlib import Path

data_dir = Path("data")

# Safe creation — no error if it already exists
data_dir.mkdir(exist_ok=True)

# Create nested dirs in one call
nested = Path("data") / "exports" / "2024"
nested.mkdir(parents=True, exist_ok=True)
```

What Happens Without `exist_ok`?

```
from pathlib import Path

data_dir = Path("data")
```

Opening Files With `pathlib`

`Path` Objects Work Directly With `open()`

```
from pathlib import Path
import json

DATA_DIR  = Path(__file__).parent / "data"
DATA_DIR.mkdir(exist_ok=True)

json_file = DATA_DIR / "contacts.json"

# Write
with open(json_file, "w") as f:
    json.dump({"contacts": []}, f, indent=2)

# Read
with open(json_file, "r") as f:
    data = json.load(f)

print(data)   # {'contacts': []}
```

Listing Directory Contents

`iterdir()` — All Entries in a Directory

```
from pathlib import Path

data_dir = Path("data")

print("Files in data/:")
for entry in data_dir.iterdir():
    print(f"  {entry.name}")   # contacts.json, backup.json, ...
```

`glob()` — Filter by Pattern

```
from pathlib import Path

data_dir = Path("data")

# All JSON files directly inside data/
json_files = list(data_dir.glob("*.json"))
```

Demo: Create `data/` and List Contents

Watch For:

- `mkdir(exist_ok=True)` called safely every run
- `Path(__file__).parent` anchors to the script location
- `iterdir()` lists what is in the folder

```
from pathlib import Path
import json

# Build paths anchored to the script
SCRIPT_DIR = Path(__file__).parent
DATA_DIR   = SCRIPT_DIR / "data"
DATA_DIR.mkdir(exist_ok=True)

# Write two sample files
for i in range(1, 3):
    p = DATA_DIR / f"sample_{i}.json"
    with open(p, "w") as f:
        json.dump({"id": i, "label": f"Sample {i}"}, f)

# List the folder
```

Lab: Data Folder — 30 min

Task

Refactor your contact manager to store `data.json` in a `data/` subdirectory, using pathlib throughout.

Step 1 — Update paths:

```
from pathlib import Path

SCRIPT_DIR = Path(__file__).parent
DATA_DIR   = SCRIPT_DIR / "data"
DATA_DIR.mkdir(exist_ok=True)

CONTACTS_FILE = DATA_DIR / "contacts.json"
```

Step 2 — Update `save_contacts` and `load_contacts`:

```
def save_contacts(contacts: list) -> None:
```

Lab: Data Folder — continued

Step 3 — List all files in `data/` on startup:

```
contacts = load_contacts()
print(f"Loaded {len(contacts)} contacts.")

print("Data folder contents:")
for entry in DATA_DIR.iterdir():
    print(f"  {entry.name}")
```

Step 4 — Run from a different directory and verify it still works:

```
# From your home directory
python /full/path/to/project/main.py
```

Completion Criteria

- [ ] `data/` folder is created automatically on first run

Common Pitfalls — Hour 43

Pitfall 1: Hard-Coded Absolute Paths

```
# Bug: only works on one specific machine
DATA_DIR = Path("/home/alice/project/data")

# Fix: anchor to the script's own location
DATA_DIR = Path(__file__).parent / "data"
```

Pitfall 2: Forgetting `exist_ok=True`

```
# Bug: crashes on the second run
DATA_DIR = Path("data")
DATA_DIR.mkdir()   # FileExistsError!

# Fix
DATA_DIR.mkdir(exist_ok=True)   # safe every time
```

Pitfall 3: Running From a Different Working Directory

Common Pitfalls — Hour 43 (continued)

Pitfall 4: Using String Concatenation for Paths

```
# Bug: breaks on Windows (backslash separator)
path = "data" + "/" + "contacts.json"   # hard-coded "/" is wrong on Windows

# Also fragile — easy to miss a slash or double up
path = "data" + "contacts.json"   # wrong — missing separator

# Fix: use Path and the "/" operator
from pathlib import Path
path = Path("data") / "contacts.json"   # always correct
```

Pitfall 5: `iterdir()` Returns `Path` Objects, Not Strings

```
from pathlib import Path

for entry in Path("data").iterdir():
    print(entry)            # PosixPath('data/contacts.json')
    print(entry.name)       # contacts.json  — the string filename
```

Quick Check — Hour 43

Exit Ticket Question: What is the difference between a relative and absolute path?

Model Answer: "A **relative path** is interpreted relative to the **current working directory (CWD)** — the folder from which the Python script is run. For example, `Path('data/contacts.json')` means 'look for a `data` folder inside wherever I am right now.' An **absolute path** starts from the root of the filesystem (e.g., `/home/user/project/data/contacts.json` on Unix, or `C:\\Users\\user\\project\\data\\contacts.json` on Windows) and means exactly one location, regardless of the CWD. The practical implication is that relative paths can break when a script is run from different directories, while absolute paths always resolve to the same file — though hard-coded absolute paths break on other machines. The best practice for application data is `Path(__file__).parent / 'data'`, which is relative to the script itself rather than to wherever the user runs it from."

Hour 44: Exception Handling (Full Basics Treatment)

Learning Outcomes

- Use `try/except` to catch and handle runtime errors
- Distinguish `try/except/else/finally` clauses
- Catch specific exceptions: `ValueError`, `FileNotFoundError`, `json.JSONDecodeError`
- Understand why catching broadly (`except Exception`) is discouraged

Why Programs Crash

Runtime Errors Are Inevitable

```
# User types "abc" when we asked for a number
age = int(input("Enter your age: "))
# ValueError: invalid literal for int() with base 10: 'abc'
# → program crashes, all unsaved data lost

# File is missing
with open("data.json", "r") as f:
    data = json.load(f)
# FileNotFoundError: [Errno 2] No such file or directory: 'data.json'
# → crash on first run before any data exists
```

Exceptions Are Not Bugs — They Are Signals

```
# An exception signals: "something went wrong — what should we do?"
# If we don't handle it → program crashes with a traceback
# If we handle it → we can show a friendly message and keep running
```

`try` / `except` — The Basic Pattern

Syntax

```
try:
    # code that might raise an exception
    value = int(input("Enter a number: "))
except ValueError:
    # runs only if a ValueError was raised inside the try block
    print("That is not a valid number. Please enter digits only.")
```

What Happens Step by Step

```
try:
    value = int("abc")     # raises ValueError
    print("After error")   # SKIPPED — never reached
except ValueError:
    print("Caught it!")    # runs instead
print("After the block")   # always runs — we recovered
```

Catching Specific Exceptions

Why Specific Is Better Than Broad

```
# Broad (discouraged at Basics level)
try:
    value = int(input("Enter a number: "))
except Exception:
    print("Something went wrong.")   # catches EVERYTHING — hides real bugs

# Specific (correct pattern)
try:
    value = int(input("Enter a number: "))
except ValueError:
    print("Please enter a whole number.")   # only catches bad conversion
```

Multiple `except` Clauses

```
try:
    with open("data.json", "r") as f:
```

`try / except / else / finally`

Full Structure

```
try:
    result = int(input("Enter a number: "))
except ValueError:
    print("Not a valid number.")
    result = None
else:
    # runs ONLY if no exception was raised in try
    print(f"You entered: {result}")
finally:
    # ALWAYS runs — exception or not
    print("Input processing complete.")
```

When to Use Each Clause

Clause	When It Runs	Typical Use

Common Exceptions to Know

The Basics Toolbox

Exception	When It Occurs	Example
`ValueError`	Conversion fails	`int("abc")`
`TypeError`	Wrong type for operation	`"a" + 1`
`IndexError`	List index out of range	`lst[99]` on a 3-item list
`KeyError`	Dict key not found	`d["missing"]`
`FileNotFoundError`	File not found on open	`open("nope.txt")`
`json.JSONDecodeError`	Invalid JSON in file	`json.load(f)` on corrupt file
`ZeroDivisionError`	Division by zero	`10 / 0`

```
# All of these can be caught with specific except clauses:
```

Wrapping Numeric Input

Robust Input Loop Pattern

```python
def get_int(prompt: str) -> int:
    """Keep asking until the user enters a valid integer."""
    while True:
        raw = input(prompt).strip()
        try:
            return int(raw)
        except ValueError:
            print(f"  '{raw}' is not a whole number — please try again.")

# Usage
age = get_int("Enter your age: ")
print(f"Age: {age}")
```

Example Interaction

```
Enter your age: abc
```

Wrapping File / JSON Load

Safe Load Pattern

```python
import json
from pathlib import Path

def load_contacts(filepath: Path) -> list:
    """Load contacts from JSON, returning [] on any file/parse error."""
    try:
        with open(filepath, "r") as f:
            data = json.load(f)
        return [contact_from_dict(d) for d in data]
    except FileNotFoundError:
        print(f"No save file found at {filepath} — starting fresh.")
        return []
    except json.JSONDecodeError:
        print(f"{filepath} appears corrupted — starting fresh.")
        return []
```

Why `else` Can Help Here

Demo: Harden the Contact Manager

Watch For:

- Each `except` catches a specific, named exception
- Friendly messages shown to the user
- Program continues after errors — no crash

```
import json
from pathlib import Path

CONTACTS_FILE = Path(__file__).parent / "data" / "contacts.json"

def safe_load(filepath: Path) -> list:
    try:
        with open(filepath, "r") as f:
            return json.load(f)
    except FileNotFoundError:
        print("No existing data — starting fresh.")
        return []
    except json.JSONDecodeError:
        print("Data file corrupted — starting fresh.")
        return []

def get_int(prompt: str) -> int:
    while True:
```

Lab: Harden Your Program — 30 min

Task

Add exception handling to your contact manager so it never crashes on bad input or missing files.

Step 1 — Wrap `load_contacts()` with `try/except`:

```
def load_contacts(filepath: Path) -> list:
    try:
        with open(filepath, "r") as f:
            data = json.load(f)
        return [contact_from_dict(d) for d in data]
    except FileNotFoundError:
        return []          # first run — no file yet
    except json.JSONDecodeError:
        print("Warning: data file is corrupted. Starting with empty list.")
        return []
```

Lab: Harden Your Program — continued

Step 3 — Replace bare `int()` calls in your menu with `get_int()`:

```
# Before (crashes on bad input)
choice = int(input("Choose option: "))

# After (safe)
choice = get_int("Choose option: ")
```

Step 4 — Test your hardening:

```
# Test 1: delete data.json — should start fresh without crashing
# Test 2: corrupt data.json (write "not json" in it) — should show warning
# Test 3: type "abc" at any numeric prompt — should show friendly message
```

Completion Criteria

- [ ] Program does not crash when `data.json` is missing

Common Pitfalls — Hour 44

Pitfall 1: Catching `Exception` Too Broadly

```
# Bug: catches EVERYTHING — including bugs you haven't fixed yet
try:
    result = risky_operation()
except Exception:
    print("Something went wrong.")   # which thing? we'll never know

# Fix: catch only what you expect
try:
    result = int(user_input)
except ValueError:
    print("Please enter a number.")
```

Pitfall 2: Swallowing Errors Without a Message

```
# Bug: silent failure — user has no idea what happened
try:
```

Common Pitfalls — Hour 44 (continued)

Pitfall 3: Trying to Use the Variable Set in `try` After an Exception

```
# Bug: value is never assigned when the exception fires
try:
    value = int(input("Enter number: "))
except ValueError:
    print("Bad input.")

print(f"You entered {value}")   # NameError if exception was raised!

# Fix: set a default before the try block
value = None
try:
    value = int(input("Enter number: "))
except ValueError:
    print("Bad input.")

if value is not None:
    print(f"You entered {value}")
```

Quick Check — Hour 44

Exit Ticket Question: Why is catching a specific exception better than catching all exceptions?

Model Answer: "Catching a **specific exception** (e.g., `except ValueError`) only intercepts the error you know how to handle — a bad type conversion — and lets all other exceptions propagate normally. This is safer for three reasons: First, unexpected bugs (like a `NameError` or `AttributeError` caused by a mistake in your code) will still crash the program with a visible traceback, making them easy to find and fix. Second, the intent is clear — a reader sees exactly which error is being handled and why. Third, you avoid accidentally masking critical errors that the program should not silently swallow. Using `except Exception` is sometimes called a 'Pokémon catch' (`catch 'em all`) — it grabs every exception, including ones you never expected, and hides them behind a generic message. Specific exception handling is one of the hallmarks of robust, maintainable Python code."

Session 11 Complete — Putting It All Together

The Full Persistence Pattern

```
from pathlib import Path
import json

# 1. Anchored paths
SCRIPT_DIR    = Path(__file__).parent
DATA_DIR      = SCRIPT_DIR / "data"
DATA_DIR.mkdir(exist_ok=True)
CONTACTS_FILE = DATA_DIR / "contacts.json"

# 2. Safe load
def load_contacts() -> list:
    try:
        with open(CONTACTS_FILE, "r") as f:
            return [contact_from_dict(d) for d in json.load(f)]
    except FileNotFoundError:
        return []
    except json.JSONDecodeError:
        print("Warning: save file corrupted — starting fresh.")
        return []

# 3. Save
def save_contacts(contacts: list) -> None:
    with open(CONTACTS_FILE, "w") as f:
        json.dump([c.to_dict() for c in contacts], f, indent=2)

# 4. Safe numeric input
def get_int(prompt: str) -> int:
    while True:
        try:
            return int(input(prompt).strip())
```

Session 11 Key Terms Reference

Term	Definition
context manager	An object that sets up and tears down resources automatically; used with `with`
`with open()`	Idiomatic Python for opening files; guarantees the file is closed on exit
file mode	`"r"` read, `"w"` write (overwrites), `"a"` append — controls how a file is opened
serialization	Converting a Python object to a storable/transmittable format (e.g., JSON)
deserialization	Reconstructing a Python object from a stored format (e.g., loading JSON)

No-Go Topics for Basics Scope

> 🚫 **Scope guardrail (Basics):** The following are **Advanced course** topics. Do not introduce them during this session.

Topic	Why It Is Advanced
Custom exception classes (`class MyError(Exception)`)	Requires inheritance understanding
Exception chaining (`raise X from Y`)	Advanced error propagation patterns
Custom context managers (`__enter__` / `__exit__`)	Advanced OOP and protocol design
`contextlib.contextmanager` decorator	Advanced generator-based context managers
Binary file I/O (`"rb"`, `"wb"`)	Encoding details, struct module —

Session 11 Common Patterns Reference

Text File — Save and Load a List

```
from pathlib import Path

def save_lines(items: list, filename: str) -> None:
    with open(filename, "w") as f:
        for item in items:
            f.write(item + "\n")

def load_lines(filename: str) -> list:
    if not Path(filename).exists():
        return []
    with open(filename, "r") as f:
        return [line.strip() for line in f if line.strip()]
```

JSON — Save and Load Objects

```
import json
```

Looking Ahead — Day 12

Next Session Builds On Today

Hour 45: Review + Mini-Project Setup

- Combine OOP (Day 10), persistence (Day 11), and previous CLI skills
- Plan and scaffold a multi-file project

Hour 46: Building the Mini-Project

- Implement the full data layer with JSON persistence
- Separate UI, data, and logic into modules

Hour 47: Testing and Debugging

- Manual testing strategies
- Reading tracebacks; using `print()` for debugging